

UNIVERSIDAD GERARDO BARRIOS



**Universidad
Gerardo Barrios**

FACULTAD DE CIENCIA Y TECNOLOGÍA

MÓDULO 4: DESARROLLO DE APLICACIONES CON IA

REPORTE TÉCNICO DE ENTREGA - SEMANA 2

"API Inteligente, Contratos de Integración y Evidencias de Prueba"

PROYECTO: EdubotAI (Tutor Virtual Inteligente)

GRUPO: Equipo 4

INTEGRANTES:

- **Gabriel Eduardo Henriquez Gonzalez**
 - **Melvin Josué Pereira Amaya**
 - **Alexis Manuel Calix Magaña**

DOCENTE: Marco Antonio Arévalo Zambrano

FECHA DE ENTREGA: Sábado 18 de julio de 2026

1. INTRODUCCIÓN Y PROPÓSITO DEL PROYECTO

EdubotAI es una solución diseñada para estudiantes de Ciencias de la Computación de la Universidad Gerardo Barrios (UGB), cuyo objetivo es combatir la deserción académica y resolver la falta de retroalimentación inmediata fuera del aula.

Durante la **Semana 2**, se ejecutó la migración del backend hacia un enfoque más robusto y escalable, estructurando una API basada en **FastAPI** que centraliza la interacción con los servicios de IA Generativa de **Groq Cloud API**. Esta capa intermedia permite independizar la lógica de negocio de los clientes (como Telegram Bot API o interfaces Web), garantizando validaciones de datos estructuradas, manejo unificado de errores y cumplimiento estricto de las mejores prácticas de seguridad en la gestión de variables de entorno y tokens de API.

2. ARQUITECTURA TÉCNICA DE LA API

La API fue programada bajo el lenguaje de programación Python utilizando el framework de alto desempeño **FastAPI**. Para asegurar la integridad de la información enviada por los clientes, se implementaron modelos de validación estrictos mediante **Pydantic**.

- **Motor Generativo:** Groq Cloud API, utilizando el modelo de última generación **llama-3.1-8b-instant**.
- **Gestión de Credenciales (Seguridad):** Se ha evitado rigurosamente la exposición de tokens sensibles. La clave de la API de Groq se extrae dinámicamente en tiempo de ejecución utilizando variables de entorno a través de la librería python-dotenv y almacenada en un archivo seguro .env que está configurado en el archivo .gitignore para prevenir su publicación accidental en repositorios de control de versiones.

3. CONTRATOS DE INTEGRACIÓN (DIFINICIÓN DE ENDPOINTS)

A. Endpoint de Estado de Salud (/health)

- **Ruta:** /health
- **Método:** GET
- **Propósito:** Monitorear que el servidor de FastAPI se encuentra activo y respondiendo solicitudes de red.
- **Payload de Entrada:** Ninguno.
- **Formato de Respuesta (JSON):**

```
{
  "status": "activo",
  "servicio": "EdubotAI API"
}
```

B. Endpoint de Metadatos de la API (/metadata)

- **Ruta:** /metadata
- **Método:** GET
- **Propósito:** Proveer información descriptiva sobre la versión del software, la tecnología utilizada y los modelos de IA empleados.
- **Payload de Entrada:** Ninguno.
- **Formato de Respuesta (JSON):**

```
{  
  "proyecto": "EdubotAI",  
  "proposito": "Proveer tutoría personalizada y predicción de deserción académica.",  
  "version": "1.0.0",  
  "tecnologias": ["FastAPI", "Groq Cloud API", "Pickle"],  
  "modelos": ["modelo_edubot.pkl", "llama-3.1-8b-instant"]  
}
```

C. Endpoint Inteligente Principal (/chat)

- **Ruta:** /chat
- **Método:** POST
- **Propósito:** Enviar la consulta académica del estudiante, validarla, procesar la solicitud con el modelo LLM y retornar la explicación generada.
- **Validaciones Aplicadas (Pydantic & FastAPI Exceptions):**
 - El campo mensaje debe contener al menos 2 caracteres (evita consultas vacías).
 - El campo modulo obligatoriamente debe ser uno de los tres módulos académicos autorizados: C++, Data Science o Web.
- **Payload de Entrada (JSON):**

```
{  
  "estudiante_id": "UGB2026",  
  "mensaje": "¿Qué es una variable en Data Science?",  
  "modulo": "Data Science"  
}
```
- **Respuesta Exitosa (HTTP 200 OK):**

```
{  
  "respuesta_ia": "(Explicación real de la IA adaptada al contexto del módulo seleccionado)",  
  "puntuacion_actualizada": 5,  
  "estado": "Success"  
}
```
- **Respuesta de Error Controlado - Módulo Inválido (HTTP 400 Bad Request):**

```
{  
  "detail": "Módulo no válido. Seleccione: C++, Data Science o Web."
```

```
}
```

- **Respuesta de Error de Validación - Mensaje muy corto (HTTP 422 Unprocessable Entity):**

```
{
  "detail": [
    {
      "loc": ["body", "mensaje"],
      "msg": "ensure this value has at least 2 characters",
      "type": "value_error.any_str.min_length"
    }
  ]
}
```

\pagebreak

4. EVIDENCIAS DE PRUEBAS DE EJECUCIÓN

Las pruebas se llevaron a cabo utilizando la interfaz interactiva automatizada **Swagger UI** provista nativamente por FastAPI en la dirección local <http://127.0.0.1:8000/docs>.

EVIDENCIA 1: Verificación de Salud de la API (/health)

- **Acción:** Petición GET realizada exitosamente al endpoint de salud para comprobar la disponibilidad del servicio.
- **Respuesta del Servidor:** {"status": "activo", "servicio": "EdubotAI API"}

The screenshot displays the Swagger UI interface for the API. At the top, the endpoint `GET /health` is selected, with a description "Verificar estado del servicio". Below this, the "Parameters" section indicates "No parameters". A blue "Execute" button is visible. The "Responses" section shows a successful response with a status code of 200. The "Response body" is displayed as a JSON object: `{ "status": "activo", "servicio": "EdubotAI API" }`. The "Response headers" section shows: `content-length: 45, content-type: application/json, date: Mon, 13 Jul 2026 02:57:51 GMT, server: unicorn`. A "Download" button is present next to the response body. At the bottom, a table with columns "Code" and "Description" is partially visible.

EVIDENCIA 2: Obtención de Metadatos de la API (/metadata)

- **Acción:** Petición GET para auditar los metadatos declarados del proyecto EdubotAI.
- **Respuesta del Servidor:** Retorna las tecnologías de desarrollo y el modelo de IA integrado de forma exitosa.

The screenshot shows a REST client interface with the following sections:

- GET /metadata** (Obtener metadatos)
- Parameters:** No parameters. A **Cancel** button is present.
- Execute** (blue button) and **Clear** (text button) buttons.
- Responses:**
 - Curl:**

```
curl -X 'GET' \
  http://127.0.0.1:8000/metadata' \
  -H 'accept: application/json'
```
 - Request URL:**

```
http://127.0.0.1:8000/metadata
```
 - Server response:**

Code	Details
200	Response body<pre>{ "proyecto": "EdubotAI", "proposito": "Proveer tutoría personalizada y predicción de deserción académica.", "version": "1.0.0", "tecnologias": ["FastAPI", "Groq Cloud API", "Pickle"], "modelos": ["modelo_edubot.pkl", "llama-3.3-70b-versatile"] }</pre> Download

EVIDENCIA 3: Interacción Inteligente Exitosa con la IA (/chat)

- **Acción:** Petición POST enviando un payload válido con consulta académica real conectada a Groq Cloud con el modelo llama-3.1-8b-instant.
- **Datos Enviados:** Estudiante UGB2026, Mensaje de prueba, Módulo Data Science.
- **Resultado:** La API procesa el mensaje de manera segura y retorna la explicación contextualizada generada de forma autónoma por la IA.

POST

/chat Ejecutar capacidad inteligente

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{  "estudiante_id": "UGB2026",  "mensaje": "¿Qué es una variable en Data Science?",  "modulo": "Data Science"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/chat' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "estudiante_id": "UGB2026",
    "mensaje": "¿Qué es una variable en Data Science?",
    "modulo": "Data Science"
  }'
```

Request URL

http://127.0.0.1:8000/chat

Server response

CodeDetails

200

Response body

```
{
  "respuesta_ia": "¡Hola! Me alegra poder ayudarte con tus dudas en Data Science.\n\n¿Qué es una variable en Data Science?*\n\nEn Data Science, una variable se refiere a una característica o atributo que se mide o se observa en un conjunto de datos. Las variables son los elementos fundamentales que componen un conjunto de datos y pueden ser de diferentes tipos, como numéricas, categóricas, temporales, entre otras.\n\n**Tipos de variables**\n\nExisten varios tipos de variables en Data Science, incluyendo:\n\n1. **Variables numéricas**: Son variables que toman valores numéricos, como la edad, el salario, la altura, etc.\n2. **Variables categóricas**: Son variables que toman valores categóricos, como el género, la nacionalidad, el color de ojos, etc.\n3. **Variables temporales**: Son variables que se refieren a fechas o horas, como la fecha de nacimiento, la hora de llegada, etc.\n4. **Variables de texto**: Son variables que contienen texto, como el nombre, la dirección, la descripción de un producto, etc.\n\n**Características de las**",
  "puntuacion_actualizada": 5,
  "estado": "Success"
}
```

Response headers

```
content-length: 1109
content-type: application/json
date: Mon, 13 Jul 2026 03:05:53 GMT
server: uvicorn
```

Responses

CodeDescription

200Successful Response

Links

No links

EVIDENCIA 4: Validación y Manejo Controlado de Errores

- **Acción:** Petición POST enviando un payload con un módulo inexistente (Matematicas) para forzar la validación de seguridad de la API.
- **Resultado:** La API deniega el procesamiento, evitando llamadas innecesarias al API de Groq y retornando un código de estado controlado HTTP 400 Bad Request indicando claramente las opciones disponibles.

POST /chat Ejecutar capacidad inteligente

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value Schema

```
{  "estudiante_id": "UGB2826",  "mensaje": "Hola, necesito ayuda con integrales.",  "modulo": "Matematicas"}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \  'http://127.0.0.1:8000/chat' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "estudiante_id": "UGB2826",  "mensaje": "Hola, necesito ayuda con integrales.",  "modulo": "Matematicas"  }'
```

Request URL

http://127.0.0.1:8000/chat

Server response

Code	Details
400	Error: Bad Request

Undocumented

Response body

```
{  "detail": "Módulo no válido. Seleccione: C++, Data Science o Web."}
```

Download

Response headers

```
content-length: 69  content-type: application/json  date: Mon, 13 Jul 2026 03:18:41 GMT  server: uvicorn
```

Responses

Code	Description	Links
------	-------------	-------

5. DIAGNÓSTICO DE CUMPLIMIENTO (RÚBRICA DE EVALUACIÓN)

A continuación, se detalla el nivel de cumplimiento de los requerimientos especificados en el descriptor de la tarea de la Semana 2:

1. **Funcionalidad Inteligente expuesta por API.** Se estructuró el endpoint principal /chat implementando el consumo en tiempo real de la API de **Groq Cloud** con el modelo actualizado y optimizado llama-3.3-70b-versatile.
2. **Contratos Claros de Entrada y Salida.** Documentado rigurosamente en el archivo docs/api.md, detallando de forma transparente métodos, tipos de datos, parámetros obligatorios, esquemas JSON y ejemplos prácticos.
3. **Validación y Manejo de Errores.** Pydantic se encarga de impedir envíos nulos o mal estructurados. El servidor implementa lógica condicional para responder con estados 400 Bad Request cuando los módulos no forman parte de las rutas académicas autorizadas, previniendo malgastar recursos.
4. **Endpoint de Salud y Metadatos.** Se crearon exitosamente las rutas /health y /metadata que retornan la salud técnica de la API y su propósito.
5. **Evidencia de Prueba y Documentación.** El repositorio cuenta con el archivo descriptivo docs/api.md, el presente informe con capturas demostrables del funcionamiento, y el archivo README.md del repositorio actualizado con guías de instalación.
6. **Seguridad e Integridad de Datos.** La clave de la API de Groq se mantiene 100% aislada utilizando configuraciones .env y .gitignore, garantizando que bajo ningún escenario las claves sean visibles de manera pública en GitHub.